

ElectroGo Platform: End-to-End Infrastructure Provisioning and Application Deployment on Microsoft Azure Using Terraform, Azure Kubernetes Service (AKS), Helm, and Azure DevOps

1. Introduction

Cloud-native applications require a reliable, scalable, and secure infrastructure that can be deployed consistently across different environments. Manual provisioning of cloud resources often results in configuration inconsistencies, increased deployment time, and operational complexity. To overcome these challenges, organizations adopt Infrastructure as Code (IaC), containerization, Kubernetes orchestration, and Continuous Integration/Continuous Deployment (CI/CD).

The ElectroGo Platform project demonstrates a complete cloud deployment solution using Microsoft Azure. Infrastructure is provisioned using Terraform, applications are deployed to Azure Kubernetes Service (AKS), secrets are securely managed using Azure Key Vault, Docker images are stored in Azure Container Registry (ACR), and Azure DevOps automates the build and deployment process. The project follows modern DevOps principles and provides a production-style deployment architecture.

2. Project Aim

The primary aim of the ElectroGo Platform project is to automate the provisioning of Azure infrastructure and deploy a containerized web application using Kubernetes and Azure DevOps. The project focuses on creating a secure, scalable, and highly available environment that minimizes manual intervention and enables continuous software delivery.

3. Project Objectives

The objectives of this project are:

- Automate Azure infrastructure provisioning using Terraform.
- Deploy Kubernetes clusters using Azure Kubernetes Service.
- Store Docker images in Azure Container Registry.
- Secure sensitive information using Azure Key Vault.
- Configure private networking for Azure Database for MySQL Flexible Server.
- Implement Helm-based Kubernetes deployments.
- Automate application deployment using Azure DevOps CI/CD pipelines.
- Follow Infrastructure as Code and DevOps best practices.

4. Project Overview

The ElectroGo Platform is an enterprise-style cloud deployment project designed to automate infrastructure provisioning and application deployment on Microsoft Azure. Terraform provisions all required cloud resources including networking, AKS, Azure Key Vault, and MySQL Flexible Server. Docker is used to containerize the application, while Azure Container Registry stores application images.

Azure Kubernetes Service hosts the application containers and provides scalability, reliability, and simplified container orchestration. Azure DevOps pipelines automate application builds, image publishing, and Kubernetes deployments using Helm charts. This approach ensures every deployment follows the same process, reducing deployment errors and improving reliability.

5. Solution Architecture

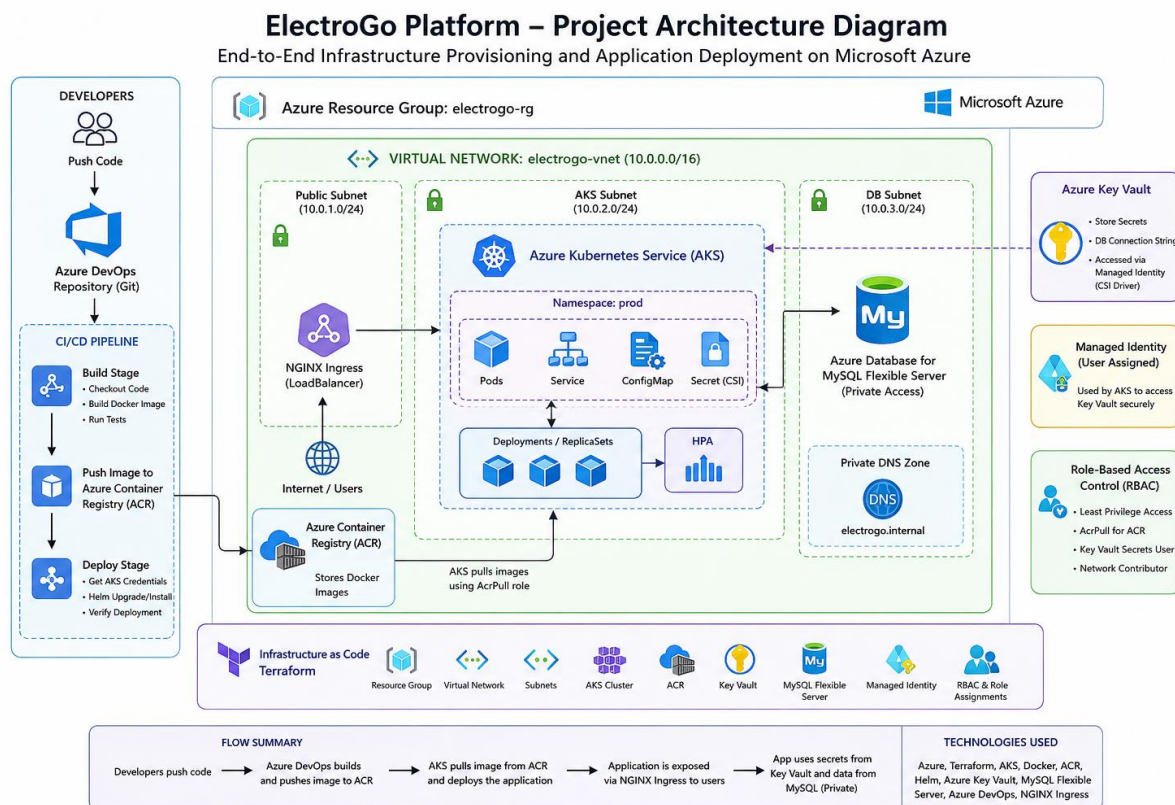
The ElectroGo Platform architecture consists of multiple Azure services working together to provide a secure deployment environment.

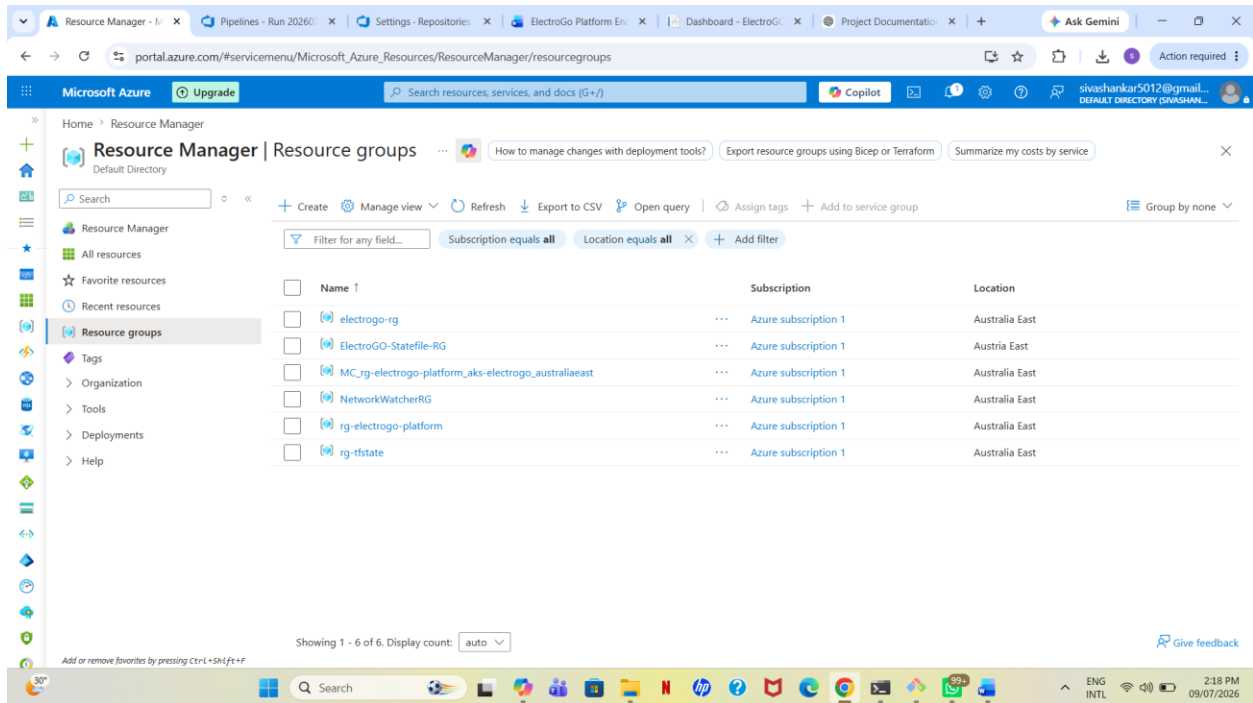
The Resource Group acts as the logical container for all Azure resources. A Virtual Network is created with dedicated subnets for AKS, MySQL Flexible Server, and future Application Gateway integration. Azure Kubernetes Service hosts the application workloads, while Azure Container Registry stores Docker images.

Azure Key Vault securely stores application secrets and database connection strings. These secrets are accessed from Kubernetes using the CSI Secrets Store Driver and Managed Identity. Azure Database for MySQL Flexible Server is deployed with private access to ensure database communication remains inside the virtual network.

Azure DevOps pipelines automatically build Docker images, push them to Azure Container Registry, and deploy them into AKS using Helm.

Screenshot Placement





6. Technology Stack

Technology	Purpose
Microsoft Azure	Cloud Platform
Terraform	Infrastructure as Code
Azure Kubernetes Service	Container Orchestration
Docker	Application Containerization
Azure Container Registry	Docker Image Repository
Azure Key Vault	Secret Management
Azure Database for MySQL Flexible Server	Database
Helm	Kubernetes Package Manager
Azure DevOps	Continuous Integration and Deployment
Git	Source Code Management
NGINX Ingress	Traffic Routing

7. Infrastructure Provisioning using Terraform

Terraform is used to provision every Azure resource automatically. The infrastructure is organized into reusable modules that improve maintainability and scalability. The deployment begins by initializing Terraform, validating configuration files, generating an execution plan, and finally applying the infrastructure.

Terraform creates the Resource Group, Virtual Network, Subnets, Network Security Groups, Managed Identity, Azure Kubernetes Service cluster, Azure Key Vault, Azure Database for MySQL Flexible Server, and required role assignments.

Using Infrastructure as Code ensures repeatable deployments while eliminating manual configuration errors.

Screenshot

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Finding hashicorp/random versions matching "~> 3.6"...
- Finding hashicorp/time versions matching "~> 0.11"...
- Finding hashicorp/azurerm versions matching "~> 3.117"...
- Finding hashicorp/kubernetes versions matching "~> 2.29"...
- Installing hashicorp/random v3.9.0...
- Installed hashicorp/random v3.9.0 (signed by HashiCorp)
- Installing hashicorp/time v0.14.0...
- Installed hashicorp/time v0.14.0 (signed by HashiCorp)
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)
- Installing hashicorp/kubernetes v2.38.0...
- Installed hashicorp/kubernetes v2.38.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> terraform validate
Success! The configuration is valid.

PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> terraform plan
Acquiring state lock. This may take a few moments...
data.azure_rm_client_config.current: Reading...
data.azure_rm_container_registry.acr: Reading...
data.azure_rm_client_config.current: Read complete after 0s [id=Y2xpZW50Q29uZmlncy9jbGllbnRJZD0wNGIwNzc5NS04ZGRiLTQ2MWEtYmJLZS0wMmY5ZTFiZjdiNDY7b2JqZWNoSWQ9NGE2N2JmNzZmM2MzMC00ODNhLWFlMGItZjYwZGY3OWE4YTVhO3N1YnNjcmLwdGlvbklkPTI3MmYzNjNjLTg1ZmEtNGY1NSliYjFkLWNlYWFmNDk0NmYwNDt0ZW5hbnRJZD03Mjc2ZmIxZC0yNDFlLTQ2MDYtYmY3YS0wMjQ1NWFlOTU4MGQ=]
data.azure_rm_container_registry.acr: Read complete after 1s [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/electrogo-rg/providers/Microsoft.ContainerRegistry/registries/electrogoacr20250630]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# azurerm_key_vault.main will be created
+ resource "azurerm_key_vault" "main" {
  + access_policy              = (known after apply)
  + enable_rbac_authorization = true
  + id                        = (known after apply)
  + location                  = "australiaeast"
  + name                      = (known after apply)
  + public_network_access_enabled = true
  + resource_group_name       = "rg-electrogo-platform"
  + sku_name                  = "standard"
  + soft_delete_retention_days = 90
  + tags                      = {}
  + "environment" = "prod"
```

```

PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> terraform apply --auto-approve
Acquiring state lock. This may take a few moments...
data.azurearm_container_registry.acr: Reading...
data.azurearm_client_config.current: Reading...
data.azurearm_client_config.current: Read complete after 0s [id=Y2xpZW50Q29uZmNlcy9jbGllbnRJZD0wNGIwNzc5NS04ZGRiLTQ2MWEtYmJlZS0wMmY5ZTFiZjdiNDY7b2JqZWNoSWQ9NGE2N2JmNzMtM2MzMC00ODNhLWFlhMGItZjYwZGY3OWE4YTVhO3N1YnNjcmLwdGlvbklkPTI3MWYzNjNiLTg1ZmEtNGY1NS1iYjFkLWNlYWVmNDk0NmYwNDt0ZW5hbnRJZD03Mjc2ZmIxZC0yNDFlLTQ2MDYtYmY3YS0wMjQ1NWFOU4MGQ=]
data.azurearm_container_registry.acr: Read complete after 0s [id=/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/electrogo-rg/providers/Microsoft.ContainerRegistry/registries/electrogoacr20250630]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# azurearm_key_vault.main will be created
+ resource "azurearm_key_vault" "main" {
+   access_policy              = (known after apply)
+   enable_rbac_authorization = true
+   id                        = (known after apply)
+   location                  = "australiaeast"
+   name                      = (known after apply)
+   public_network_access_enabled = true
+   resource_group_name       = "rg-electrogo-platform"
+   sku_name                  = "standard"
+   soft_delete_retention_days = 90
+   tags                      = {
+     "environment" = "prod"
+     "managed_by"  = "terraform"
+   }
}

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
agic_subnet_id = "/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/rg-electrogo-platform/providers/Microsoft.Network/virtualNetworks/vnet-electrogo/subnets/snet-electrogo-agic"
aks_cluster_name = "aks-electrogo"
aks_kube_config_host = <sensitive>
aks_node_resource_group = "MC_rg-electrogo-platform_aks-electrogo_australiaeast"
aks_resource_group = "rg-electrogo-platform"
key_vault_name = "kv-electrogo-yxkrvr"
key_vault_secrets_provider_client_id = "5a95c86f-b519-4c7a-b960-078c75e2fcde"
mysql_database_name = "electrogo"
mysql_fqdn = "mysql-electrogo-prod-dmy6qs.mysql.database.azure.com"
oidc_issuer_url = "https://australiaeast.oic.prod-aks.azure.com/7276fb1d-241b-4606-bf7a-02455aa9580d/be3ce6ed-e8e6-4b91-bfc0-f94440e28cbe/"
vnet_id = "/subscriptions/271f363b-85fa-4f55-bb1d-ceaaf4946f04/resourceGroups/rg-electrogo-platform/providers/Microsoft.Network/virtualNetworks/vnet-electrogo"

```

8. Azure Networking

A dedicated Virtual Network provides secure communication between Azure resources. Multiple subnets are created to separate Kubernetes workloads, database services, and future ingress components.

The MySQL Flexible Server uses private networking to prevent public database access. Network Security Groups protect each subnet by allowing only required traffic.

This architecture enhances security while maintaining efficient communication between Azure services.

Screenshot Placement

Microsoft Azure portal showing the configuration of a virtual network named **vnet-electrogo**.

Overview

Resource group: **rg-electrogo**
Location: **Australia East**
Subscription: **Azure subscription 1**
Subscription ID: **271f363b-85fa-4f55-bb1d-ceaaf4946f04**

Address space: **10.0.0.0/16**
Advertised gateway pr...: **-**
Subnets: **4 subnets**
DNS servers: **168.63.129.16**
BGP community string: **Configure**
Virtual network ID: **c2347df0-addrb-4a85-8016-2e9b62654239**

Tags: **environment: prod**, **managed_by: terraform**

Capabilities

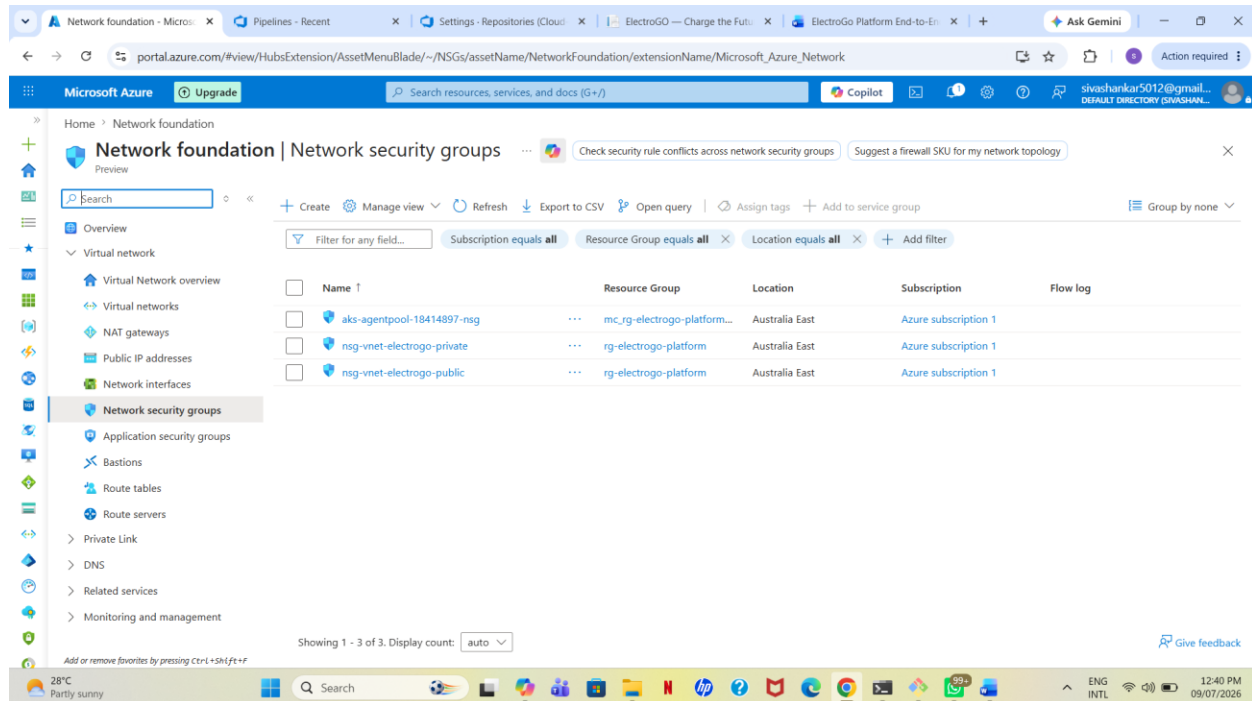
- DDoS protection**: Not configured
- Azure Firewall**: Not configured
- Peerings**: Not configured

Subnets

Create subnets to segment the virtual network address space into smaller ranges for use by your applications. When you deploy resources into a subnet, Azure assigns the resource an IP address from the subnet.

Name	IPv4	IPv6	Available IPs	Delegated to	Security group	Route table
snet-electrogo-agic	10.0.3.0/24	-	251	-	-	-
snet-electrogo-aks	10.0.16.0/20	-	4,033	-	-	-
snet-electrogo-public	10.0.1.0/24	-	251	-	nsg-vnet-e...	-
snet-electrogo-private	10.0.2.0/24	-	251	Microsoft...	nsg-vnet-e...	-

Showing 4 subnets



9. Azure Kubernetes Service (AKS)

Azure Kubernetes Service hosts the ElectroGo application inside Kubernetes pods. The cluster consists of system and user node pools that separate Kubernetes system components from application workloads.

AKS automatically manages cluster operations such as node provisioning, upgrades, scaling, and high availability. Helm charts simplify deployment and application upgrades.

Screenshot

aks-electrogo - Node pools

Node pools provide space for applications to run. Node pools of different types can be added to the cluster to handle a variety of workloads, existing node pools can be scaled and upgraded, or node pools that are no longer needed can be deleted. Each node pool will contain nodes backed by virtual machines. [Learn more about node pools](#)

Autoscale events 0 Autoscale warnings 0 Scale-up not triggered 0

Node pool	Provisioning state	Power state	Scale method	Target nodes	Ready nodes	Autoscaling status	Mode	Operating system
userpool	Succeeded	Running	Autoscale	1	1	No scale activity	User	Ubuntu Linux
systempool	Succeeded	Running	Autoscale	1	1	No scale activity	System	Ubuntu Linux

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
aks-systempool-20460849-vmss000000 Ready     <none>   4h25m v1.35.5
aks-userpool-28327041-vmss000000   Ready     <none>   4h18m v1.35.5
```

userpool | Nodes

Node name: All Add label filter

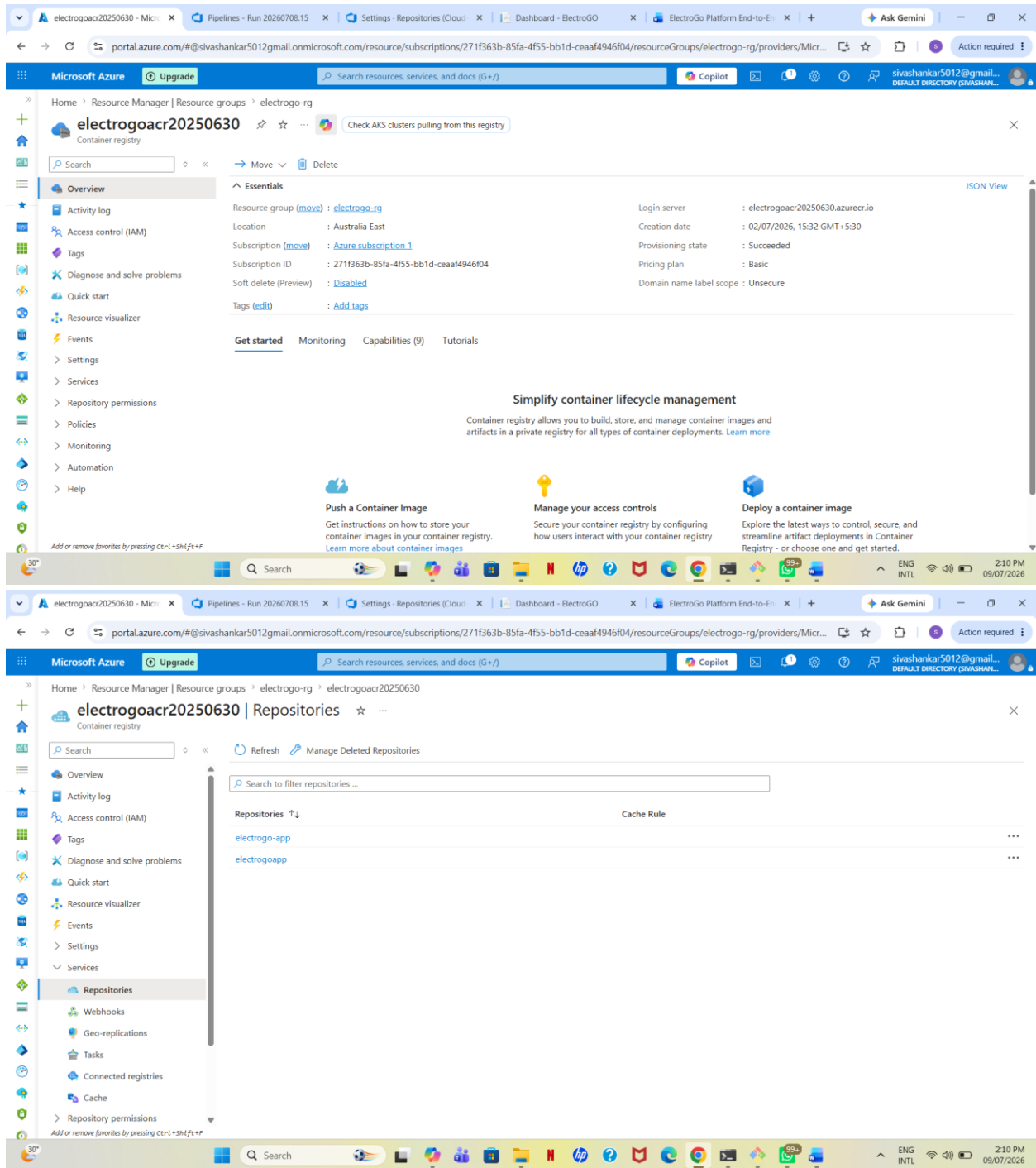
Node	Status	CPU	Memory	Disk	Pods	Kubernetes version
aks-userpool-28327041-vmss000000	Ready	6%	22%	18%	13	1.35.5

10. Azure Container Registry

Azure Container Registry stores Docker images built by Azure DevOps pipelines. After each successful build, the application image is pushed into ACR.

The AKS cluster is granted the AcrPull role, allowing Kubernetes nodes to securely pull images during deployment.

Screenshot



11. Azure Key Vault

Azure Key Vault securely stores sensitive information such as database connection strings.

Instead of storing credentials inside Kubernetes Secrets, the application retrieves them dynamically using the Azure Key Vault CSI Driver and Managed Identity. This improves security and simplifies secret management.

Screenshot

The first screenshot shows the 'Overview' page of the Key Vault 'kv-electrogo-yxkrvr'. The left sidebar lists navigation options: Overview, Activity, Access, Tags, Diagnose, Access policies, Resource visualizer, Events, Objects, Settings, Monitoring, Automation, and Help. The main content area displays 'Essentials' with details: Resource group (rg-electrogo-platform), Location (Australia East), Subscription (Azure subscription 1), and Subscription ID (271f363b-85fa-4f55-bb1d-ceaaf4946f04). It also shows the Vault URI, Sku (Standard), Directory ID, Directory Name, Soft-delete (Enabled), and Purge protection (Disabled). Tags include environment: prod, managed_by: terraform, and project: electrogo. Below this, there are three cards: 'Control access to key vault', 'Enable logging and set up alerts', and 'Turn on recovery options'. The second screenshot shows the 'Secrets' page of the same Key Vault. The left sidebar is updated to include 'Keys', 'Secrets', and 'Certificates'. The main content area shows a table of secrets with columns: Name, Type, Status, and Expiration date. One secret is listed: 'db-connection-string' with status 'Enabled'.

Name	Type	Status	Expiration date
db-connection-string		✓ Enabled	

12. Application Deployment using Helm

Helm is used to package and deploy the application into Kubernetes.

The Helm chart contains Kubernetes Deployment, Service, Ingress, ConfigMap, and SecretProviderClass resources.

Using Helm allows version-controlled deployments, easy rollbacks, and environment-specific configuration.

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> helm upgrade --install ingress-nginx ingress-nginx/ingress-nginx `
>> --namespace ingress-nginx --create-namespace `
>> --set controller.replicaCount=1 `
>> --set controller.resources.requests.cpu=100m `
>> --set controller.resources.requests.memory=128Mi `
>> --wait --timeout 5m
Release "ingress-nginx" does not exist. Installing it now.
NAME: ingress-nginx
LAST DEPLOYED: Wed Jul  8 18:15:53 2026
NAMESPACE: ingress-nginx
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
The ingress-nginx controller has been installed.
It may take a few minutes for the load balancer IP to be available.
You can watch the status by running 'kubectl get service --namespace ingress-nginx ingress-nginx-controller --output wide --watch'

An example Ingress that makes use of the controller:
  apiVersion: networking.k8s.io/v1
  kind: Ingress
  metadata:
    name: example
    namespace: foo
  spec:
    ingressClassName: nginx
    rules:
      - host: www.example.com
```

13. Azure DevOps CI/CD Pipeline

Azure DevOps automates the application deployment lifecycle.

The Build stage retrieves source code, builds a Docker image, and pushes it to Azure Container Registry.

The Deploy stage connects to Azure Kubernetes Service, installs or upgrades the Helm release, verifies deployment status, and exposes the application.

Automatic pipeline execution occurs whenever code is pushed to the main branch.

Screenshot

Network foundation - Micro... x ElectroGo-Kubernetes.git - P... x Settings - Repositories (Cloud... x ElectroGo — Charge the Futu... x ElectroGo Platform End-to-E... x + Ask Gemini

dev.azure.com/sivashankar5012/Cloud-Forge/_apps/hub/ms.vss-build-web.ci-designer-hub?pipelineId=48&branch=main

Azure DevOps sivashankar5012 / Cloud-Forge / Pipelines

Cloud-Forge +

- Overview
- Boards
- Repos
- Pipelines
- Pipelines
- Environments
- Releases
- Library
- Task groups
- Deployment groups
- Test Plans
- Artifacts
- Project settings

ElectroGo-Kubernetes.git

main ElectroGo-Kubernetes.git / pipelines/azure-pipelines-cd.yml

```

1 trigger:
2   branches:
3     include:
4       - main
5
6 pool:
7   vmImage: 'ubuntu-latest'
8
9 variables:
10  - acrName: 'electrogoacr20250630'
11  - imageRepo: 'electrogo-app'
12  - imageTag: '$(build.BuildId)'
13  - k8sNamespace: 'prod'
14  - helmChartPath: 'k8s/helm/electrogo-chart'
15  - azureSubscription: 'sc-azure-electrogo'
16  - resourceGroup: 'rg-electrogo-platform'
17  - clusterName: 'aks-electrogo'
18
19 stages:
20   - stage: 'Build & Push image to ACR'
21     - stage: Build
22       - job: 'Build & Push docker Image'
23         - job: BuildAndPush
24           - displayname: 'Build and push to ACR'
25             - steps:
26               - checkout: self
27               - checkout: self
28

```

Tasks

- .NET Core
- Android signing
- Ant
- App Center distribute
- App Center test
- Archive files
- ARM template deployment
- Azure App Configuration Export

28°C Partly sunny

Azure DevOps sivashankar5012 / Cloud-Forge / Pipelines / ElectroGo-Kubernetes.git / 20260708.15

Cloud-Forge +

- Overview
- Boards
- Repos
- Pipelines
- Pipelines
- Environments
- Releases
- Library

Jobs in run #20260708.15

- Build & Push Docker Image
- Build and push to ACR 48s
- Initialize job 4s
- Checkout ElectroGo... 2s
- Build and push ima... 40s
- Post-job: Checkout ... <1s
- Finalize Job <1s

Helm upgrade/install

```

1 Pool: Azure Pipelines
2 Image: ubuntu-latest
3 Agent: Azure Pipelines 1
4 Started: Yesterday at 22:04
5 Duration: 1m 32s
6
7 > Condition evaluation

```

Azure DevOps sivashankar5012 / Cloud-Forge / Pipelines / ElectroGo-Kubernetes.git / 20260708.15

Cloud-Forge +

- Overview
- Boards
- Repos
- Pipelines
- Pipelines
- Environments
- Releases
- Library
- Task groups
- Deployment groups
- Test Plans
- Artifacts

Jobs in run #20260708.15

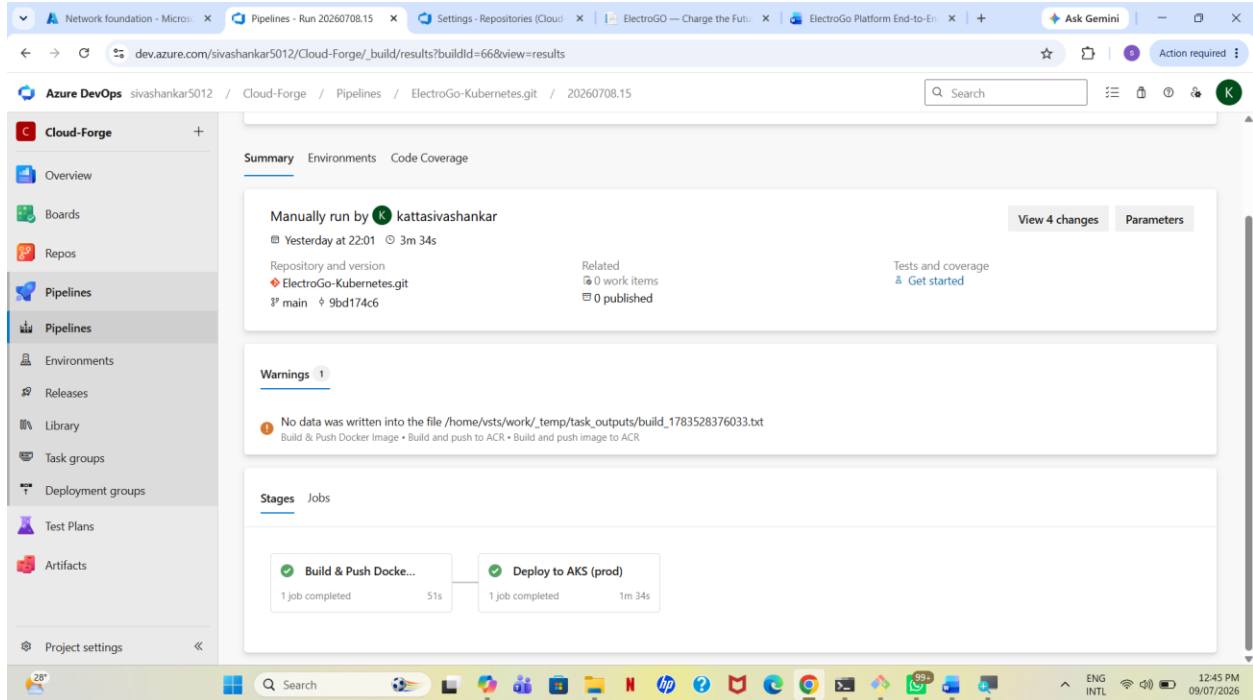
- Post-job: Checkout ... <1s
- Finalize Job <1s
- Deploy to AKS (prod)
- Helm upgrade/inst... 1m 32s
- Initialize job 4s
- Checkout ElectroGo... 2s
- Install Helm 2s
- Get AKS credentials 10s
- Install NGINX Ingres... 49s
- Helm upgrade/insta... 18s
- Post-deploy verificati... 5s
- Post-job: Checkout ... <1s
- Finalize Job <1s

Post-job: Checkout ElectroGo-Kubernetes.git@main to s

```

1 > Condition evaluation
2 Starting: Checkout ElectroGo-Kubernetes.git@main to s
3
4 =====
5 Task          : Get sources
6 Description    : Get sources from a repository. Supports Git, TfsVC, and SVN repositories.
7 Version       : 1.0.0
8 Author        : Microsoft
9 Help          : [More Information](https://go.microsoft.com/fwlink/?linkid=798199)
10
11 =====
12 Cleaning any cached credential from repository: ElectroGo-Kubernetes.git (Git)
13 Finishing: Checkout ElectroGo-Kubernetes.git@main to s

```



14. Security Implementation

Security is implemented using multiple Azure services.

Managed Identity eliminates hardcoded credentials.

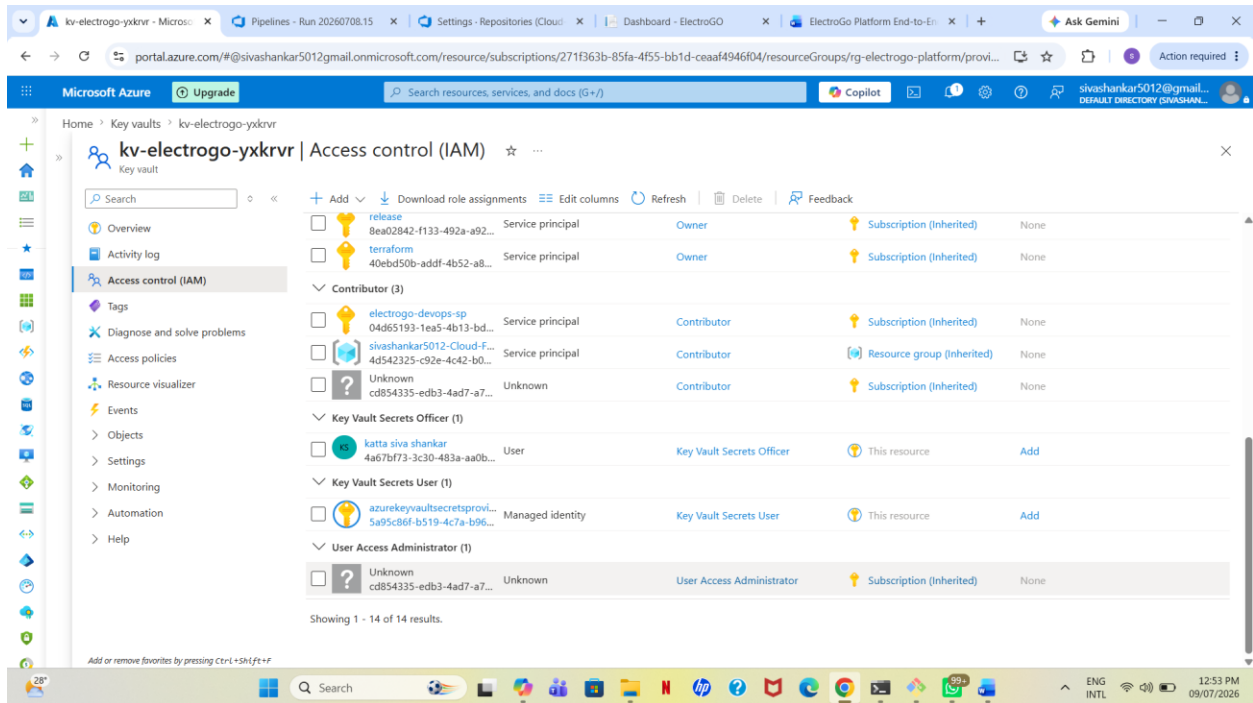
Azure Key Vault stores sensitive secrets.

Role-Based Access Control ensures least privilege access.

Private networking secures database communication.

Terraform automates security configuration consistently across environments.

Screenshot



15. Testing and Validation

After deployment, Kubernetes resources are verified using kubectl commands.

Application health is confirmed by checking node status, running pods, Kubernetes services, ingress resources, and application accessibility through the external IP.

Testing confirms successful deployment and infrastructure functionality.

Screenshot Placement

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
aks-systempool-20460849-vmss000000 Ready     <none>    4h25m v1.35.5
aks-userpool-28327041-vmss000000   Ready     <none>    4h18m v1.35.5

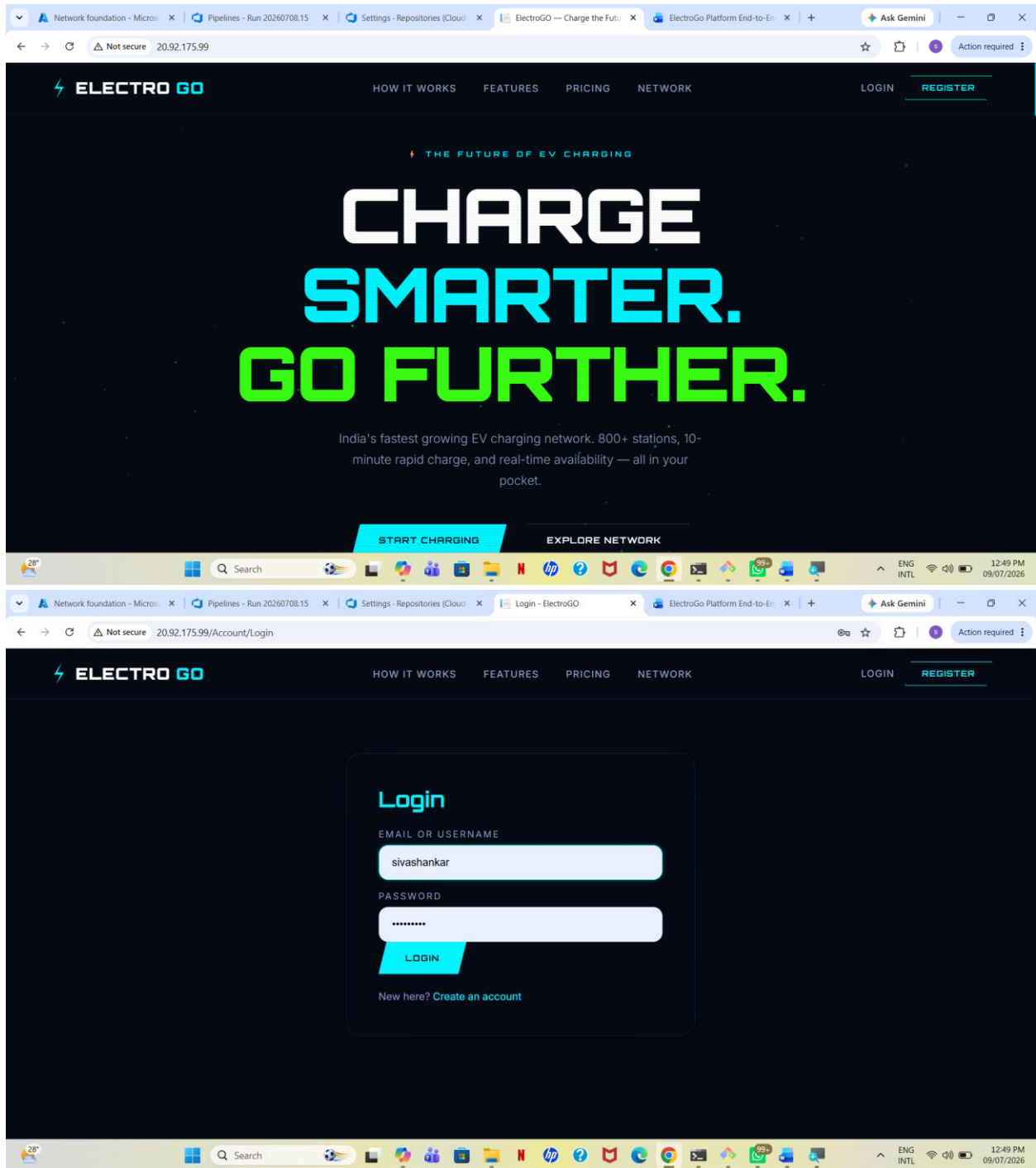
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> kubectl get pods -n prod -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP              NODE                                NOMINATE
electrogo-7988c8869c-qvvht         1/1      Running   0           71s    10.0.16.39      aks-userpool-28327041-vmss000000   <none>

PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> kubectl get svc -n prod
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
electrogo-clusterip                 ClusterIP           10.240.213.104 <none>         80/TCP           3h54m
electrogo-loadbalancer               LoadBalancer       10.240.215.15  4.147.138.2    80:31502/TCP     3h54m

PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> kubectl get svc electrogo-loadbalancer -n prod
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
electrogo-loadbalancer               LoadBalancer       10.240.215.15  4.147.138.2    80:31502/TCP     3h55m

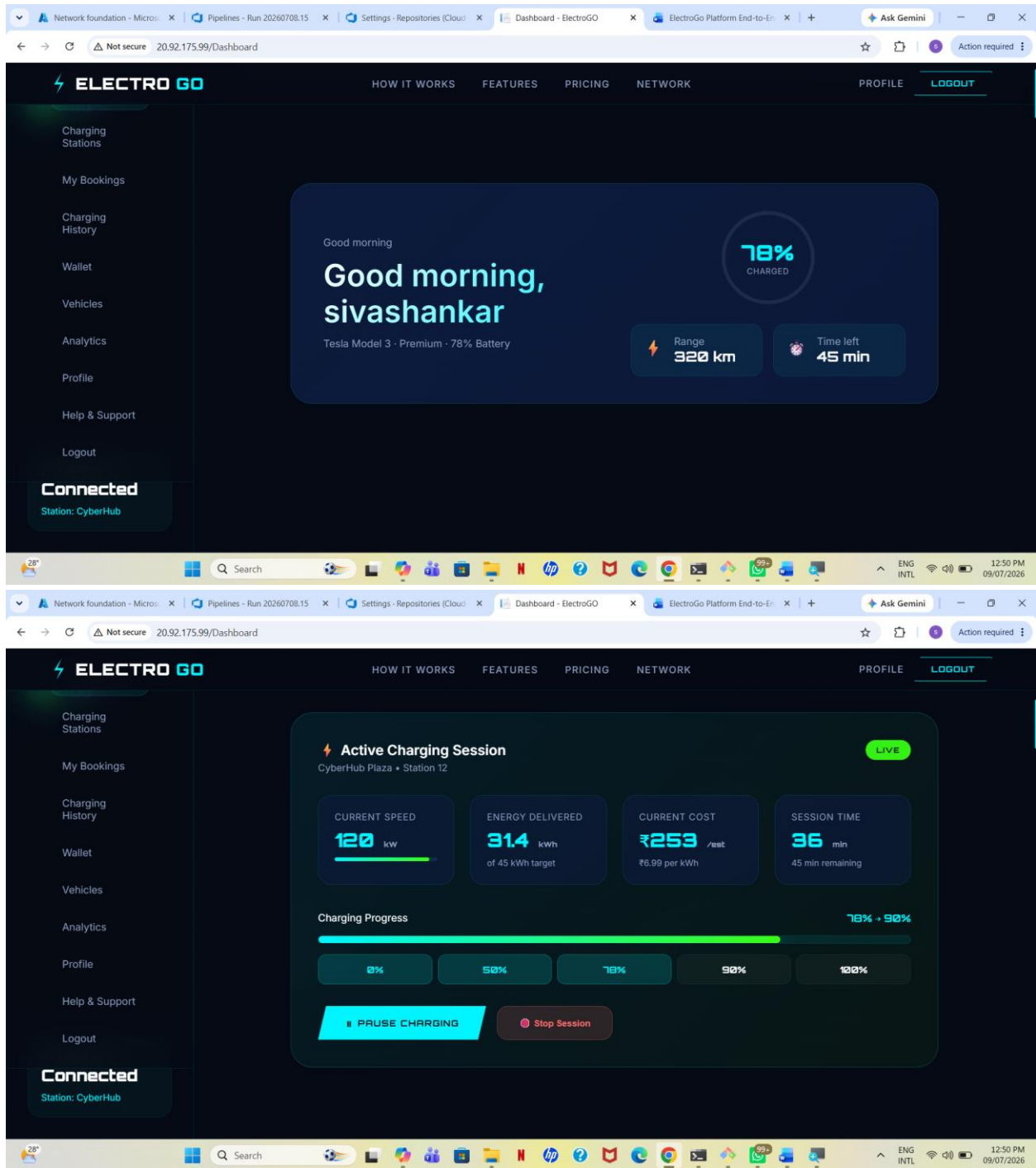
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform> kubectl get svc -n ingress-nginx ingress-nginx-controller
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
ingress-nginx-controller             LoadBalancer       10.240.20.101  20.92.175.99   80:31917/TCP,443:31447/TCP 3h52m

PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-platform>
```



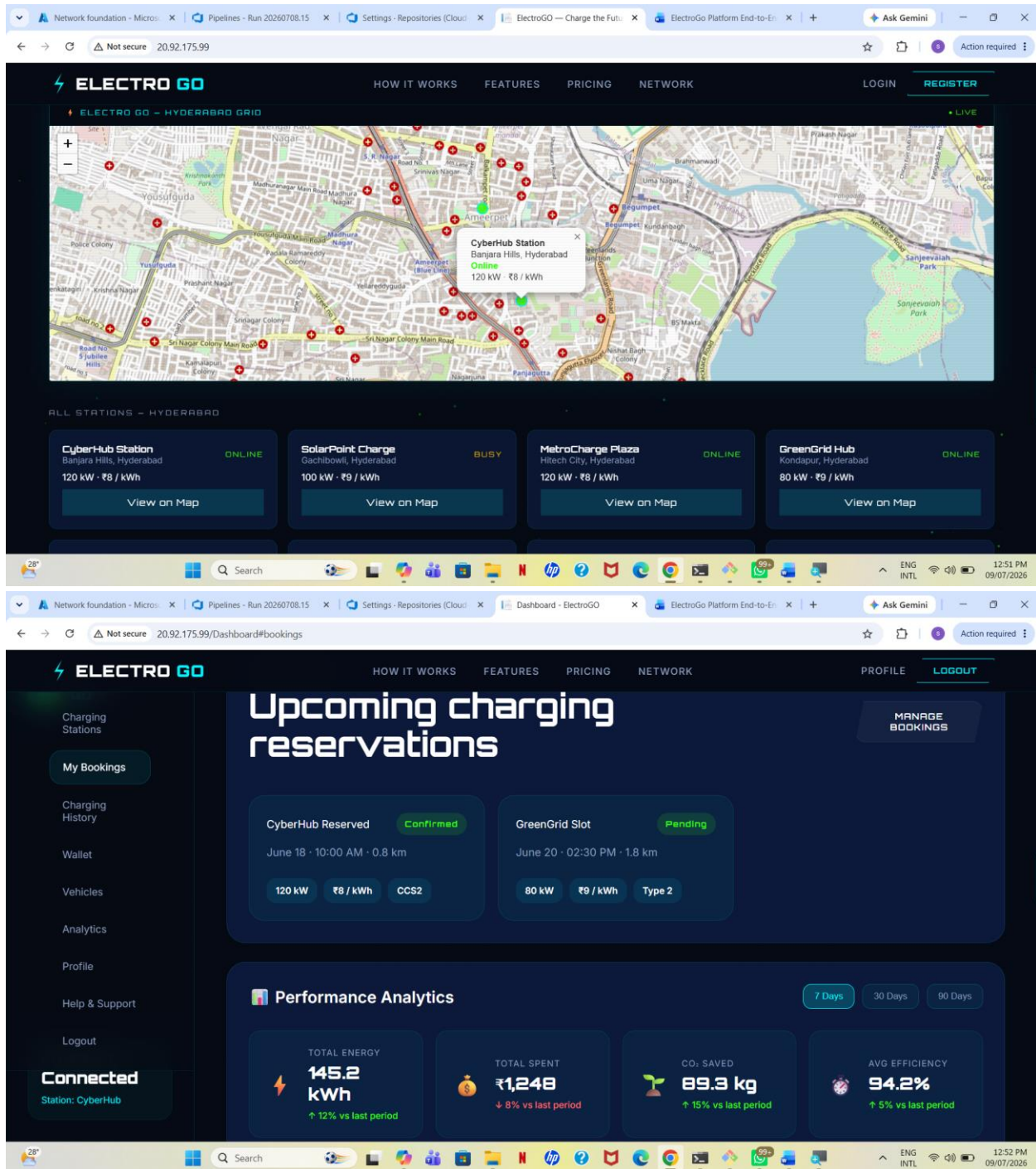
Katta siva shankar

Sivashankar9929@gmail.com



Katta siva shankar

Sivashankar9929@gmail.com



16. Challenges Faced

During implementation several challenges were encountered, including RBAC permission propagation delays, Kubernetes version compatibility issues, image pull authorization, and Azure Key Vault access configuration.

These issues were resolved by verifying role assignments, updating supported Kubernetes versions, configuring AcrPull permissions correctly, and validating Managed Identity integration.

17. Best Practices

- Infrastructure managed entirely through Terraform.
- Modular Terraform architecture.
- Remote Terraform state.
- Secure secret management using Azure Key Vault.
- Managed Identity authentication.
- Helm-based Kubernetes deployments.
- Automated CI/CD pipeline.
- Principle of Least Privilege using RBAC.
- Private networking for database services.

18. Project Outcome

The project successfully delivers a fully automated cloud-native deployment platform on Microsoft Azure. Infrastructure provisioning, Kubernetes deployment, Docker image management, secret handling, and application delivery are completely automated.

The implementation demonstrates modern DevOps practices and provides a scalable, secure, and production-ready deployment architecture suitable for enterprise applications.

Screenshot Placement

Resource Manager - Microsoft | Pipelines - Run 20260708.15 | Settings - Repositories (Cloud) | Dashboard - ElectroGO | ElectroGo Platform End-to-En | + | Ask Gemini | - | X

portal.azure.com/#servicemenu/Microsoft_Azure_ResourceManager/resourcegroups

Microsoft Azure Upgrade Search resources, services, and docs (G+J) Copilot Sivasankar5012@gmail... DEFAULT DIRECTORY (SIVASHANKAR...)

Home > Resource Manager

Resource Manager | Resource groups

Default Directory

Search

+ Create Manage view Refresh Export to CSV Open query Assign tags Add to service group Group by none

Filter for any field... Subscription equals all Location equals all Add filter

☐	Name	Subscription	Location
☐	electrogo-rg	Azure subscription 1	Australia East
☐	ElectroGO-Statefile-RG	Azure subscription 1	Austria East
☐	MC_rg-electrogo-platform_aks-electrogo_austrialeast	Azure subscription 1	Australia East
☐	NetworkWatcherRG	Azure subscription 1	Australia East
☐	rg-electrogo-platform	Azure subscription 1	Australia East
☐	rg-tfstate	Azure subscription 1	Australia East

Showing 1 - 6 of 6. Display count: auto Give feedback

Add or remove favorites by pressing Ctrl+Shift+F

Resource Manager - Microsoft | Pipelines - Run 20260708.15 | Settings - Repositories (Cloud) | Dashboard - ElectroGO | ElectroGo Platform End-to-En | + | Ask Gemini | - | X

dev.azure.com/sivashankar5012/Cloud-Forge/_build/results?buildId=66&view=results

Azure DevOps sivashankar5012 / Cloud-Forge / Pipelines / ElectroGo-Kubernetes.git / 20260708.15

Cloud-Forge Overview Boards Repos Pipelines Pipelines Environments Releases Library Task groups Deployment groups Test Plans Artifacts Project settings

Summary

Manually run by kattasivashankar

Yesterday at 22:01 3m 34s

Repository and version ElectroGo-Kubernetes.git main 9bd174c6

Related 0 work items 0 published

Tests and coverage Get started

View 4 changes Parameters

Warnings 1

No data was written into the file /home/vsts/work/_temp/task_outputs/build_1783528376033.txt

Build & Push Docker Image • Build and push to ACR • Build and push image to ACR

Stages

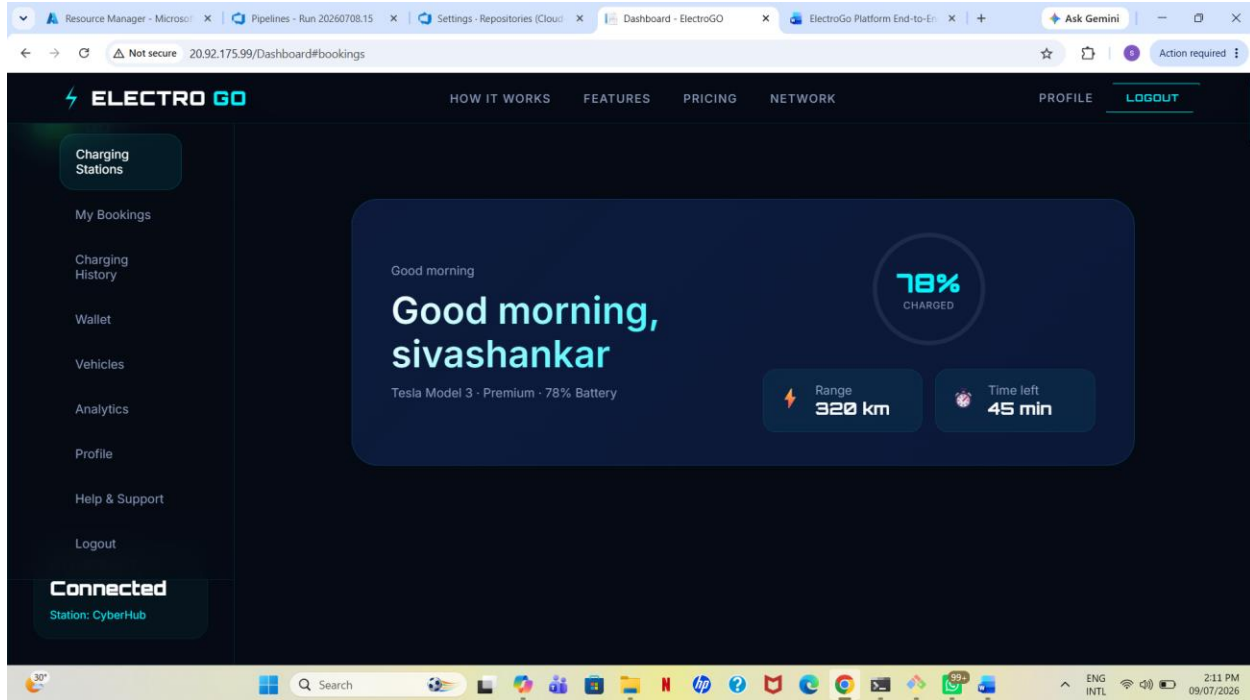
Jobs

Build & Push Docker Image 1 job completed 51s

Deploy to AKS (prod) 1 job completed 1m 34s

Katta siva shankar

Sivashankar9929@gmail.com



19. Future Enhancements

Future improvements include integrating Azure Monitor, Log Analytics, Application Insights, Application Gateway Ingress Controller (AGIC), GitOps using Argo CD, automated SSL certificate management, horizontal pod autoscaling, and multi-environment deployment pipelines.

20. Conclusion

The ElectroGo Platform project demonstrates the successful implementation of Infrastructure as Code, Kubernetes orchestration, and DevOps automation on Microsoft Azure. By integrating Terraform, Azure Kubernetes Service, Azure Container Registry, Azure Key Vault, MySQL Flexible Server, Helm, and Azure DevOps, the solution provides a secure, scalable, and fully automated deployment process.

The project eliminates manual infrastructure provisioning, improves deployment consistency, strengthens security through managed identities and secret management, and enables rapid application delivery through continuous integration and continuous deployment pipelines. Overall, the ElectroGo Platform serves as a practical example of a modern cloud-native application deployment architecture aligned with industry best practices.